

The goal of this assignment is to perform a systematic investigation of a few deep learning methods in the context of preprocessing tasks and benchmark these methods against classical methods. This assignment has been divided into 3 parts. In the first part, we will build various deep-learning models to predict the Genre of the song from the lyrics. For this, we will have to use the data on Kaggle ([Link](#)). We will train and build models in a training notebook and will later evaluate the results of these models in a demo notebook. The second part of this assignment is to develop Transfer Learning Models. We will predict the Artist of the song based on the Lyrics. We will use the best-performing models from Part 1 and retrain these models to predict the artist. We will also develop models from Scratch and compare the results with the transfer-learned models. In the third part, we will use the best-performing model to generate Lyrics.

Part 1: Lyric Genre Prediction Task

We have built Various models in Part 1 of this assignment. We will discuss the pre-processing approach and then we will compare the performances of the various models. *Please note that only 5% of the data was used for building the models as it was taking a lot of time to build models using the entire data. The reason for this is to explore more on understanding and building various deep learning methods and concepts and focus less time on building models with more data to get better accuracies.*

The following steps were taken in the process of preprocessing.

- Import Libraries and Mount Google Drive
- Loading Google Drive as a Shareable link so that we can access the dataset and files
- Loading train and test data
- Analyzing the data using functions like `.shape()`, `.head()`, `.info()` etc.
- We take only 5% of the data to build the models.
- Dropping null and checking duplicate values in the dataset.
- Split the data into training and validation.
- Preprocessing the Train, test, and validation data to get it ready for model input.
- Converting the Lyrics data to documentTermMatrix using TF-IDF
- Converting the Genre Data to Label encoding to One Hot encoding.
- Saving the train test and validation data.

Model built, trained and tested.

1. RNN

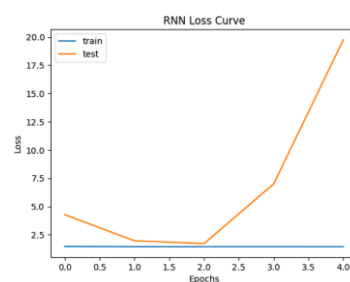
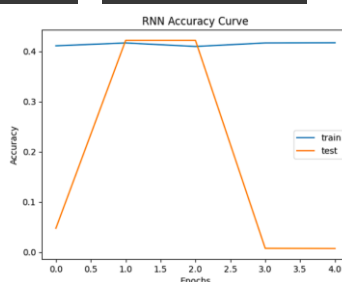
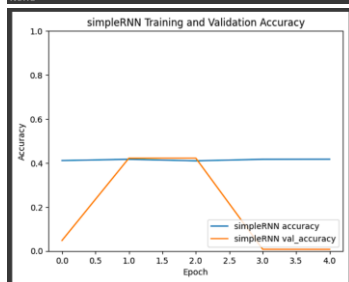
We define and train a text classification model using a Simple Recurrent Neural Network (RNN) with Keras. The model includes an embedding layer to process text data, a SimpleRNN layer for sequential analysis, batch normalization for stability, and a softmax output layer for multi-class classification. The training uses the Adam optimizer with a learning rate schedule and early stopping for improved convergence. The model is trained on TF-IDF encoded text data for a specified number of epochs and batch size.

Model: "sequential_6"

Layer (type)	Output Shape	Param #
embedding_6 (Embedding)	(None, 5000, 128)	640000
simple_rnn_2 (SimpleRNN)	(None, 128)	32896
batch_normalization_6 (Batch Normalization)	(None, 128)	512
leaky_re_lu_6 (LeakyReLU)	(None, 128)	0
flatten_6 (Flatten)	(None, 128)	0
dense_6 (Dense)	(None, 10)	1290

Total params: 674,698
 Trainable params: 674,442
 Non-trainable params: 256

Test set
Loss: 11.908
Accuracy: 0.110



2. LSTM

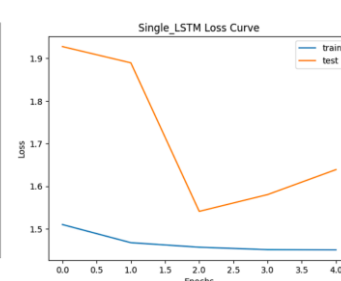
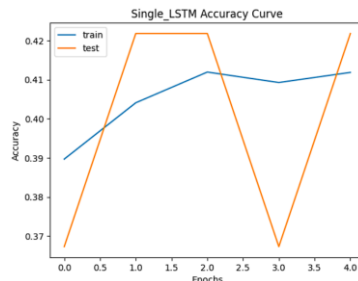
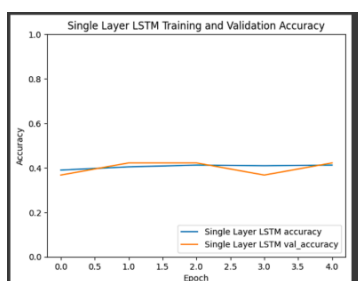
We construct and train a text classification model with a single-layer LSTM using Keras. The model includes an embedding layer for text input, a one-layer LSTM for sequence processing, a dropout layer to prevent overfitting, batch normalization for stability, LeakyReLU activation for non-linearity, and a softmax output layer for multi-class classification. The model is compiled with the Adam optimizer and categorical cross-entropy loss. It is trained on TF-IDF encoded text data with early stopping based on validation loss, using a specified batch size and number of epochs.

Model: "sequential_7"

Layer (type)	Output Shape	Param #
embedding_7 (Embedding)	(None, 5000, 128)	640000
lstm_8 (LSTM)	(None, 128)	131584
dropout (Dropout)	(None, 128)	0
batch_normalization_7 (Batch Normalization)	(None, 128)	512
leaky_re_lu_7 (LeakyReLU)	(None, 128)	0
flatten_7 (Flatten)	(None, 128)	0
dense_7 (Dense)	(None, 10)	1290

Total params: 773,386
 Trainable params: 773,130
 Non-trainable params: 256

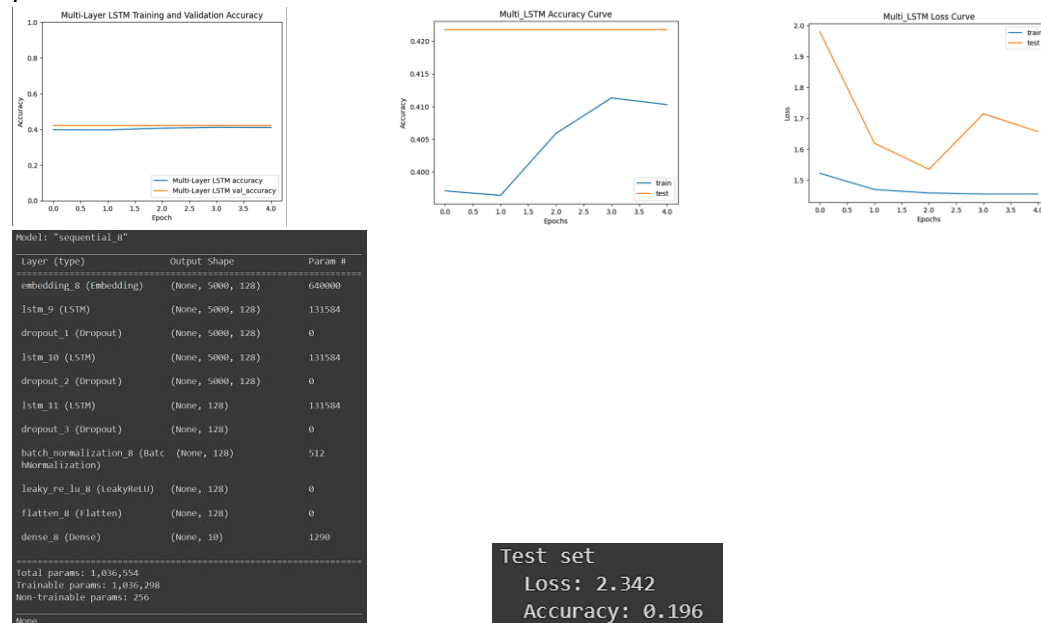
Test set
Loss: 2.347
Accuracy: 0.196



3. Multi-layer LSTM

We build and train a text classification model using a multi-layer LSTM architecture with Keras. The model includes an embedding layer for text input, followed by three LSTM layers with 128 units each, all returning sequences. Dropout layers with a dropout rate of 0.5 are added after each LSTM layer to mitigate overfitting. Batch normalization is applied for stabilization, LeakyReLU activation introduces non-linearity, and a final softmax output layer handles multi-

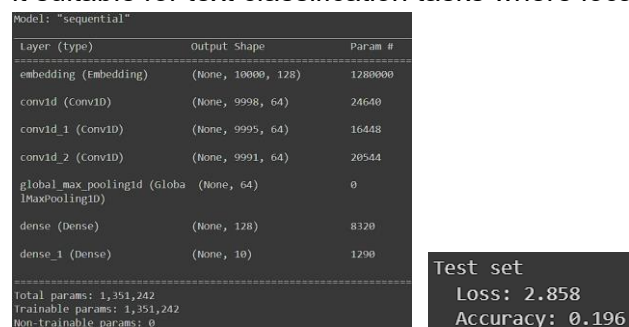
class classification. The model is compiled with the Adam optimizer and categorical cross-entropy loss. It is trained on TF-IDF encoded text data, and early stopping based on validation loss is implemented. The specified batch size and number of epochs control the training process.

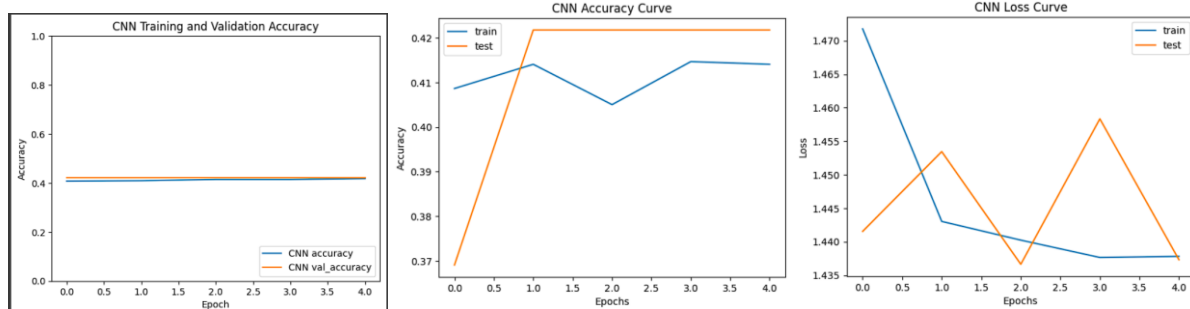


4. CNN

We have constructed and trained a text classification model with a CNN architecture using Keras. The model begins with an embedding layer to process text input. Three one-dimensional convolutional layers (Conv1D) follow, each with a different kernel size (3, 4, and 5) and ReLU activation functions. These layers are designed to capture local patterns in the input text data. A global max-pooling layer is then applied to capture the most important features from the convolutional layers. This is followed by two dense layers with ReLU activation, and the final dense layer uses softmax activation for multi-class classification. The model is compiled with the Adam optimizer and categorical cross-entropy loss. It is trained on TF-IDF encoded text data, with early stopping based on validation loss. The specified batch size and number of epochs control the training process.

The CNN architecture is designed to detect and learn hierarchical patterns in text data, making it suitable for text classification tasks where local and global features are important.





5. CNN as an Additional Layer to LSTM

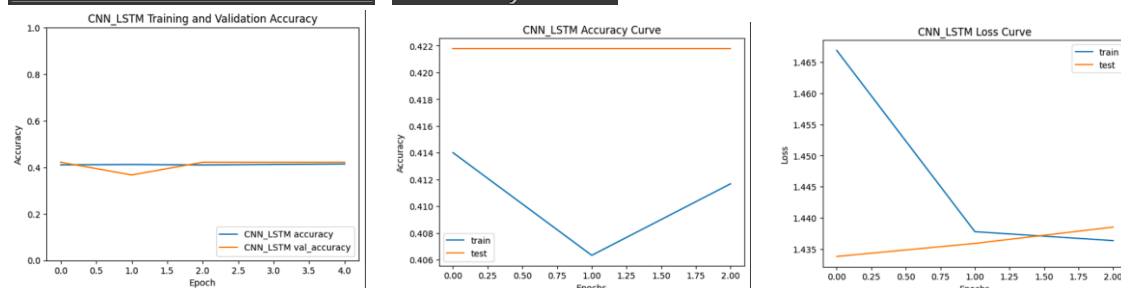
This code constructs and trains a text classification model that combines CNN and LSTM layers using Keras. The model begins with an embedding layer to process text input. Three one-dimensional convolutional layers (Conv1D) follow, each with a different kernel size (3, 4, and 5) and ReLU activation functions. These layers capture local patterns in the input text data. A global max-pooling layer is then applied to capture the most important features from the convolutional layers. Here's where it gets interesting: a Reshape layer is used to flatten the output, making it suitable for input to an LSTM layer. A one-layer LSTM with 128 units is added to capture sequential dependencies in the data. Finally, a dense layer with softmax activation is used for multi-class classification. The model is compiled with the Adam optimizer and categorical cross-entropy loss. It is trained on TF-IDF encoded text data, with early stopping based on validation loss. The specified batch size and number of epochs control the training process. This architecture combines the ability of CNNs to capture local features and LSTMs to capture sequential dependencies, making it suitable for text classification tasks where both local and global information are important.

Model: "sequential_1"

Layer (type)	Output Shape	Param #
embedding_1 (Embedding)	(None, 10000, 128)	1280000
conv1d_3 (Conv1D)	(None, 9998, 64)	28640
conv1d_4 (Conv1D)	(None, 9995, 64)	16448
conv1d_5 (Conv1D)	(None, 9991, 64)	20544
global_max_pooling1d_1 (GlobalMaxPooling1D)	(None, 64)	0
reshape (Reshape)	(None, 1, 64)	0
lstm (LSTM)	(None, 128)	98816
dense_2 (Dense)	(None, 10)	1290

Total params: 1,441,738
Trainable params: 1,441,738
Non-trainable params: 0

Test set
Loss: 2.927
Accuracy: 0.196

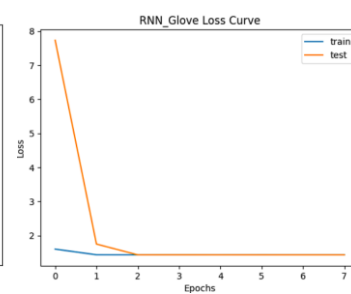
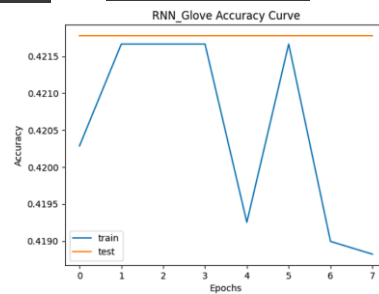
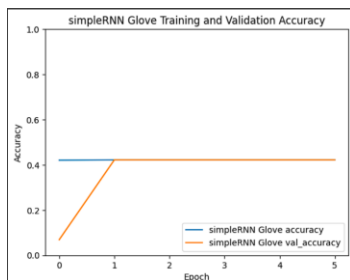


6. Pre-trained word-embedded RNN

This model prepares text data for classification by using pre-trained GloVe word embeddings. It creates an embedding matrix with pre-trained word vectors and builds a Simple RNN model for text classification. The model includes an embedding layer with fixed pre-trained weights, followed by a SimpleRNN layer, batch normalization, activation functions, and a dense softmax output layer. The model is trained on TF-IDF encoded text data with early stopping to prevent overfitting. This approach enhances the model's performance by leveraging pre-trained word embeddings to capture word meanings and context in text data.

Model: "sequential_9"		
Layer (type)	Output Shape	Param #
embedding_9 (Embedding)	(None, 5000, 100)	500000
simple_rnn_3 (SimpleRNN)	(None, 128)	29312
batch_normalization_9 (Batch Normalization)	(None, 128)	512
leaky_re_lu_9 (LeakyReLU)	(None, 128)	0
flatten_9 (Flatten)	(None, 128)	0
dense_9 (Dense)	(None, 10)	1290
Total params: 531,114		
Trainable params: 30,858		
Non-trainable params: 500,256		
None		

Test set
Loss: 2.964
Accuracy: 0.196

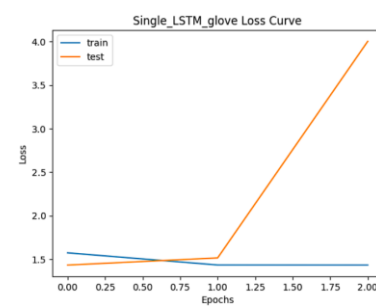
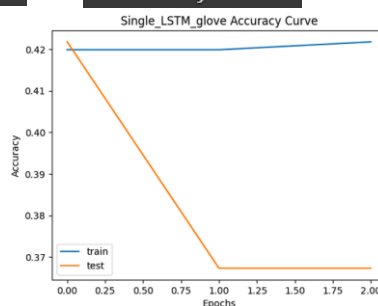
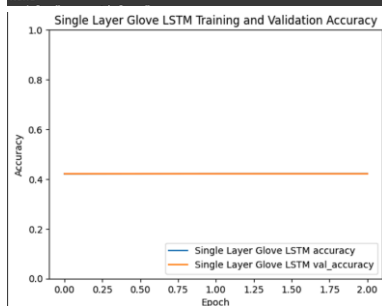


7. Pre-trained word-embedded LSTM

The model uses pre-trained GloVe word embeddings to build and train a single-layer LSTM text classification model. It tokenizes text data, prepares an embedding matrix with GloVe vectors, and constructs the model with an embedding layer initialized by these vectors. The LSTM layer is followed by batch normalization, activation functions, and a dense softmax output layer. The model is compiled using the Adam optimizer with early stopping and is trained on TF-IDF encoded text data. This approach capitalizes on pre-trained embeddings to improve model performance and mitigate overfitting.

Model: "sequential_10"		
Layer (type)	Output Shape	Param #
embedding_10 (Embedding)	(None, 5000, 100)	500000
lstm_12 (LSTM)	(None, 128)	117248
batch_normalization_10 (Batch Normalization)	(None, 128)	512
leaky_re_lu_10 (LeakyReLU)	(None, 128)	0
flatten_10 (Flatten)	(None, 128)	0
dense_10 (Dense)	(None, 10)	1290
Total params: 619,050		
Trainable params: 118,794		
Non-trainable params: 500,256		
None		

Test set
Loss: 3.094
Accuracy: 0.196



8. Pre-trained word-embedded Multilayer-LSTM

This model uses pre-trained GloVe word embeddings to build and train a multi-layer LSTM text classification model. It tokenizes text data, creates an embedding matrix with GloVe vectors, and constructs the model. The model consists of an embedding layer with locked pre-trained weights, followed by three LSTM layers with different configurations. Batch normalization, activation functions, and a softmax output layer are added. The model is compiled with Adam optimizer and categorical cross-entropy loss, trained on TF-IDF encoded

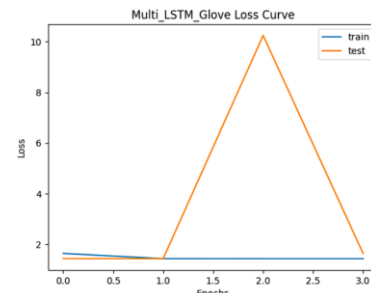
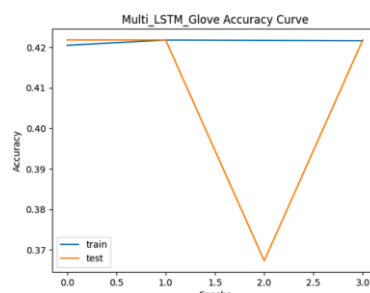
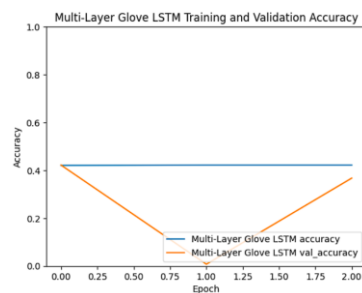
text data with early stopping to prevent overfitting. Pre-trained embeddings enhance model performance by capturing word meanings and context.

Model: "sequential_11"

Layer (type)	Output Shape	Param #
embedding_11 (Embedding)	(None, 5000, 100)	500000
lstm_13 (LSTM)	(None, 5000, 128)	117248
lstm_14 (LSTM)	(None, 5000, 64)	49408
lstm_15 (LSTM)	(None, 64)	33024
batch_normalization_11 (Batch Normalization)	(None, 64)	256
leaky_relu_11 (LeakyReLU)	(None, 64)	0
flatten_11 (Flatten)	(None, 64)	0
dense_11 (Dense)	(None, 10)	650

Total params: 700,586
 Trainable params: 200,458
 Non-trainable params: 500,128

Test set
Loss: 3.231
Accuracy: 0.152



9. Pre-trained word-embedded CNN

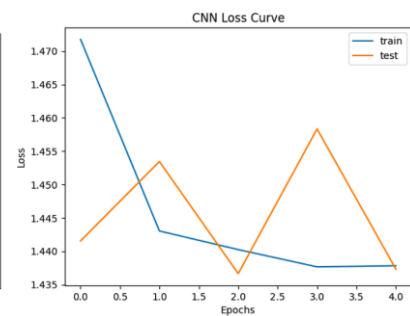
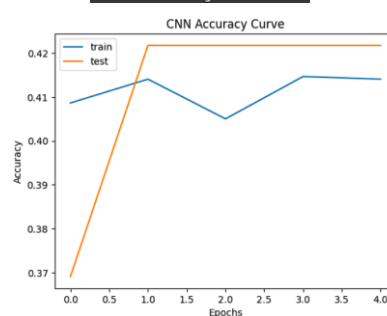
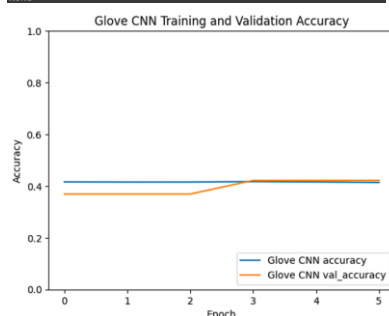
This model utilizes pre-trained GloVe word embeddings to build and train a Convolutional Neural Network (CNN) for text classification. It tokenizes the text data, creates an embedding matrix with GloVe vectors, and constructs the model with an embedding layer having locked pre-trained weights. The model includes convolutional layers with varied kernel sizes, global max-pooling, dense layers, and softmax for classification. It's compiled using the Adam optimizer, employs early stopping, and trains on TF-IDF encoded text data. This approach enhances model performance by capturing word meanings and context while mitigating overfitting.

Model: "sequential_10"

Layer (type)	Output Shape	Param #
embedding_10 (Embedding)	(None, 10000, 100)	1000000
conv1d_18 (Conv1D)	(None, 9998, 64)	19264
conv1d_19 (Conv1D)	(None, 9995, 64)	16448
conv1d_20 (Conv1D)	(None, 9991, 64)	20544
global_max_pooling1d_6 (Global Max Pooling1D)	(None, 64)	0
dense_12 (Dense)	(None, 128)	8320
dense_13 (Dense)	(None, 10)	1290

Total params: 1,065,866
 Trainable params: 65,866
 Non-trainable params: 1,000,000

Test set
Loss: 3.027
Accuracy: 0.196



10. Pre-trained word-embedded CNN as an Additional Layer to LSTM

This code builds a text classification model using both Convolutional Neural Network (CNN) and Long Short-Term Memory (LSTM) layers with pre-trained GloVe word embeddings. It tokenizes text data, creates an embedding matrix with GloVe vectors, and constructs the model. The model consists of an embedding layer with locked pre-trained weights, followed by CNN layers for local feature extraction and an LSTM layer for capturing sequential dependencies. The model is compiled using Adam optimizer and categorical cross-entropy loss and is trained with early stopping. This approach combines CNN and LSTM to improve text classification while leveraging GloVe embeddings to capture word meanings and context.

```
Model: "sequential_11"
```

Layer (Type)	Output Shape	Param #
embedding_11 (Embedding)	(None, 10000, 100)	10000000
conv1d_21 (Conv1D)	(None, 9998, 64)	19264
conv1d_22 (Conv1D)	(None, 9995, 64)	16448
conv1d_23 (Conv1D)	(None, 9991, 64)	20544
global_max_pooling1d_7 (GlobalMaxPooling1D)	(None, 64)	0
reshape_3 (Reshape)	(None, 1, 64)	0
lstm_7 (LSTM)	(None, 128)	98816
dense_14 (Dense)	(None, 10)	1290

```

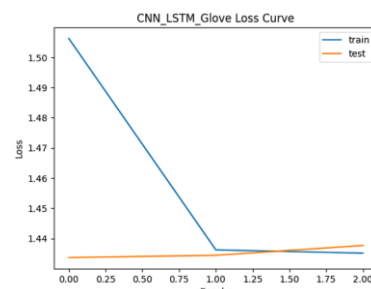
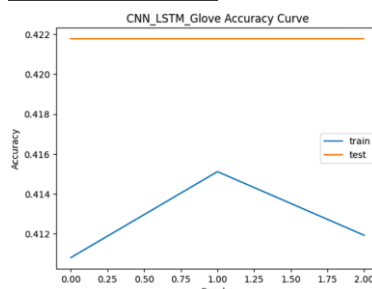
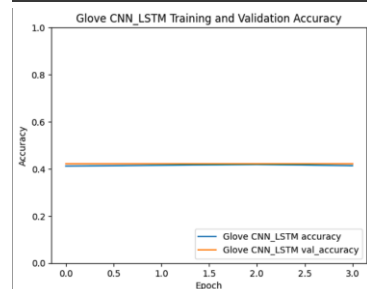
Total params: 1,156,362
Trainable params: 156,362
Non-trainable params: 1,000,000
None

```

```

Test set
Loss: 2.871
Accuracy: 0.196

```



Part 2: Lyric Genre Prediction Task

We have built various models in Part 2 of this assignment. We will discuss the pre-processing approach and then we will compare the performances of the various models.

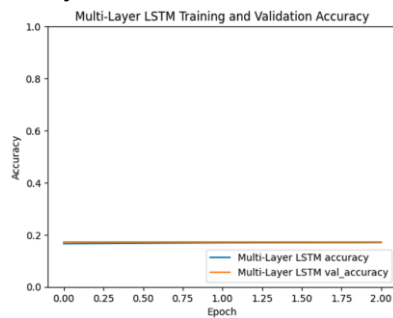
The following steps were taken in the process of preprocessing.

- Import Libraries and Mount Google Drive
- Loading Google Drive as a Shareable link so that we can access the dataset and files
- Loading train and test data
- Analyzing the data using functions like `.shape()`, `.head()`, `.info()` etc.
- Combine the train and test datasets along the rows
- Dropping null and checking duplicate values in the dataset.
- Filtering out the data for the top 10 artists in the dataset.
- Split the data into training and validation.
- Preprocessing the Train and validation data to get it ready for model input.
- Converting the Lyrics data to documentTermMatrix using TF-IDF
- Converting the Artist Data to Label encoding to One Hot encoding.
- Saving the train and validation data.

Model built, trained and tested.

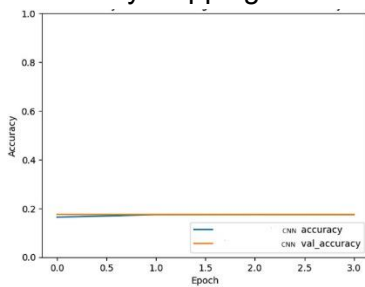
11. Transfer Learning Multilayer-LSTM

This model loads a pre-trained multi-layer LSTM text classification model and fine-tunes it on new data. It uses early stopping for model training and records the training history for further analysis.



12. Transfer Learning CNN

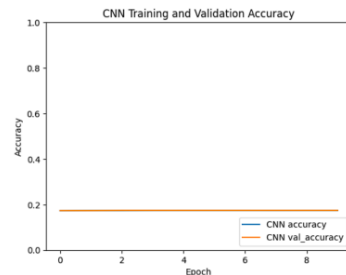
This model loads a pre-trained CNN text classification model and fine-tunes it on new data. It uses early stopping for model training and records the training history for further analysis.



13. CNN Artist Prediction

We will use the same CNN model that we built in part 1. We will only change the input and the output data to the model.

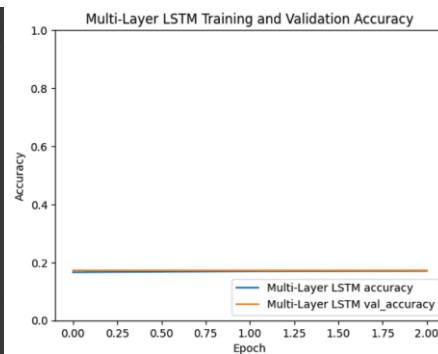
```
Model: "sequential_1"
Layer (type)                Output Shape              Param #
-----
embedding_1 (Embedding)     (None, 5000, 128)        640000
conv1d (Conv1D)              (None, 4998, 64)         24544
conv1d_1 (Conv1D)            (None, 4995, 64)         16448
conv1d_2 (Conv1D)            (None, 4991, 64)         20544
global_max_pooling1d (GlobalMaxPooling1D) (None, 64)                0
dense_1 (Dense)              (None, 128)               8320
dense_2 (Dense)              (None, 10)                1290
Total params: 711,242
Trainable params: 711,242
Non-trainable params: 0
```



14. Multilayer-LSTM Artist Prediction

We will use the same Multilayer-LSTM model that we built in part 1. We will only change the input and the output data to the model.

```
Model: "sequential"
Layer (type)                Output Shape              Param #
-----
embedding (Embedding)       (None, 5000, 128)        640000
lstm (LSTM)                  (None, 5000, 128)        131584
dropout (Dropout)            (None, 5000, 128)         0
lstm_1 (LSTM)                 (None, 5000, 128)        131584
dropout_1 (Dropout)           (None, 5000, 128)         0
lstm_2 (LSTM)                 (None, 128)               131584
dropout_2 (Dropout)           (None, 128)                0
batch_normalization (BatchNormalizatio (None, 128)                512
leaky_re_lu (LeakyReLU)      (None, 128)                0
flatten (Flatten)            (None, 128)                0
dense (Dense)                (None, 10)                1290
Total params: 1,036,954
Trainable params: 1,036,954
Non-trainable params: 256
```



Part 3: Writing your own lyrics

For this part of the assignment, we have used the same Multilayer-LSTM architecture to build a model as part 1. This model is then used to generate lyrics in this case. Firstly let us discuss the data processing steps involved.

- Import Libraries and Mount Google Drive
- Loading Google Drive as a Shareable link so that we can access the dataset and files
- Loading train data
- Analyzing the data using functions like `.shape()`, `.head()`, `.info()` etc.
- Dropping null and checking duplicate values in the dataset.
- Filtering out the data for the top 3 artists in the dataset.
- Split the data into training and validation.
- Preprocessing the Train and validation data to get it ready for model input.
- Converting the Lyrics data to documentTermMatrix using TF-IDF
- Converting the Artist Data to Label encoding to One Hot encoding.
- Saving the train and validation data.

15. Multilayer-LSTM for Lyrics Generation

Model Architecture: It defines a multi-layer LSTM model for text generation. The model architecture includes embedding layers, multiple LSTM layers, dropout layers, batch normalization, leaky ReLU activation, flattening, and a dense layer with a softmax activation.

Model Compilation: The model is compiled using the Adam optimizer and categorical cross-entropy as the loss function. It also tracks accuracy as a metric.

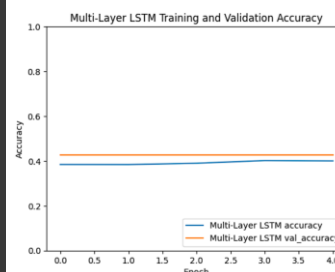
Early Stopping: An early stopping criteria is defined to monitor the validation loss and stop training if it doesn't improve for a specified number of epochs (patience).

Training: The model is trained on the provided data (`X_train_tfidf_dense`, `y_train`) using a specified batch size and number of epochs. Training progress is stored in `multi_layer_lstm_gen_history`.

Plotting Training History: The code includes a function `plot_history` to plot the training and validation accuracy of the model.

Text Generation: After training, the model is used to generate text based on seed text related to three artists: Elvis Presley, Chris Brown, and Elvis Costello. It generates lyrics for each artist using a loop and the trained model.

```
Model: "sequential_2"
Layer (type) Output Shape Param #
-----
embedding_2 (Embedding) (None, 5000, 128) 640000
lstm_6 (LSTM) (None, 5000, 128) 131584
dropout_6 (Dropout) (None, 5000, 128) 0
lstm_7 (LSTM) (None, 5000, 128) 131584
dropout_7 (Dropout) (None, 5000, 128) 0
lstm_8 (LSTM) (None, 128) 131584
dropout_8 (Dropout) (None, 128) 0
batch_normalization_2 (Batch Normalization) (None, 128) 512
leaky_relu_2 (LeakyReLU) (None, 128) 0
flatten_2 (Flatten) (None, 128) 0
dense_2 (Dense) (None, 3) 387
Total params: 1,035,493
Trainable params: 1,035,395
Non-trainable params: 256
None
Generated elvis presley song : The song lyrics is related to the artist of elvis presley
Generated chris brown song : The song lyrics is related to the artist of chris brown
Generated elvis costello song : The song lyrics is related to the artist of elvis costello
```



Model Comparison

Below we compare the different models that we have built, the training and the validation accuracies graphs as well as the test results accuracies and loss curves.

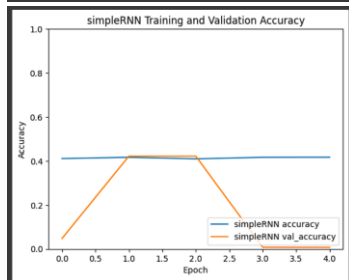
1. RNN vs LSTM

Model: "sequential_6"

Layer (type)	Output Shape	Param #
embedding_6 (Embedding)	(None, 5000, 128)	640000
simple_rnn_2 (SimpleRNN)	(None, 128)	32896
batch_normalization_6 (Batch Normalization)	(None, 128)	512
leaky_re_lu_6 (LeakyReLU)	(None, 128)	0
flatten_6 (Flatten)	(None, 128)	0
dense_6 (Dense)	(None, 10)	1290

Total params: 674,698
 Trainable params: 674,442
 Non-trainable params: 256

None



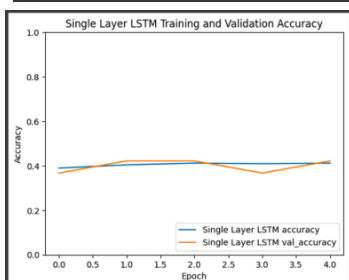
Test set
 Loss: 11.908
 Accuracy: 0.110

Model: "sequential_7"

Layer (type)	Output Shape	Param #
embedding_7 (Embedding)	(None, 5000, 128)	640000
lstm_8 (LSTM)	(None, 128)	131584
dropout (Dropout)	(None, 128)	0
batch_normalization_7 (Batch Normalization)	(None, 128)	512
leaky_re_lu_7 (LeakyReLU)	(None, 128)	0
flatten_7 (Flatten)	(None, 128)	0
dense_7 (Dense)	(None, 10)	1290

Total params: 773,386
 Trainable params: 773,130
 Non-trainable params: 256

None



Test set
 Loss: 2.347
 Accuracy: 0.196

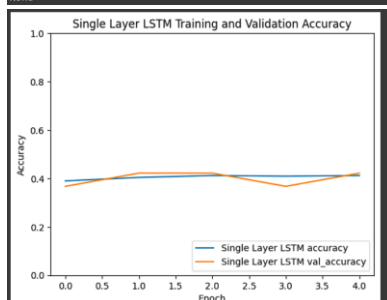
2. LSTM vs Multi-layer LSTM

Model: "sequential_7"

Layer (type)	Output Shape	Param #
embedding_7 (Embedding)	(None, 5000, 128)	640000
lstm_8 (LSTM)	(None, 128)	131584
dropout (Dropout)	(None, 128)	0
batch_normalization_7 (Batch Normalization)	(None, 128)	512
leaky_re_lu_7 (LeakyReLU)	(None, 128)	0
flatten_7 (Flatten)	(None, 128)	0
dense_7 (Dense)	(None, 10)	1290

Total params: 773,386
 Trainable params: 773,130
 Non-trainable params: 256

None



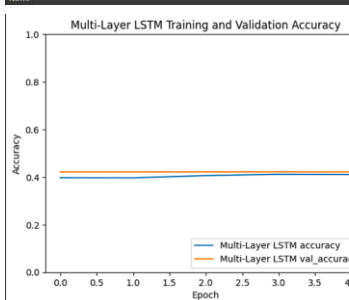
Test set
 Loss: 11.908
 Accuracy: 0.110

Model: "sequential_8"

Layer (type)	Output Shape	Param #
embedding_8 (Embedding)	(None, 5000, 128)	640000
lstm_9 (LSTM)	(None, 5000, 128)	131584
dropout_1 (Dropout)	(None, 5000, 128)	0
lstm_10 (LSTM)	(None, 5000, 128)	131584
dropout_2 (Dropout)	(None, 5000, 128)	0
lstm_11 (LSTM)	(None, 128)	131584
dropout_3 (Dropout)	(None, 128)	0
batch_normalization_8 (Batch Normalization)	(None, 128)	512
leaky_re_lu_8 (LeakyReLU)	(None, 128)	0
flatten_8 (Flatten)	(None, 128)	0
dense_8 (Dense)	(None, 10)	1290

Total params: 1,036,554
 Trainable params: 1,036,298
 Non-trainable params: 256

None

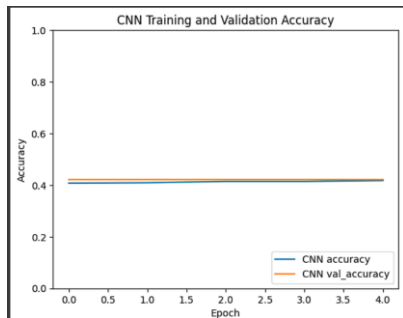


Test set
 Loss: 2.342
 Accuracy: 0.196

3. CNN vs. CNN as an additional layer to LSTM

Model: "sequential"		
Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 10000, 128)	12800000
conv1d (Conv1D)	(None, 9998, 64)	24640
conv1d_1 (Conv1D)	(None, 9995, 64)	16448
conv1d_2 (Conv1D)	(None, 9991, 64)	20544
global_max_pooling1d (GlobalMaxPooling1D)	(None, 64)	0
dense (Dense)	(None, 128)	8320
dense_1 (Dense)	(None, 10)	1290

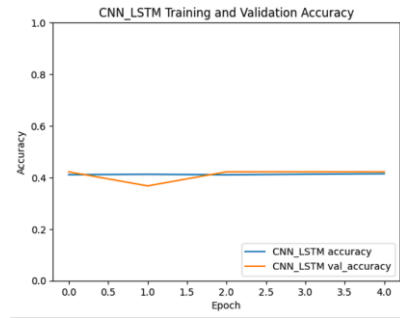
Total params: 1,351,242
 Trainable params: 1,351,242
 Non-trainable params: 0



Test set
 Loss: 2.858
 Accuracy: 0.196

Model: "sequential_1"		
Layer (type)	Output Shape	Param #
embedding_1 (Embedding)	(None, 10000, 128)	12800000
conv1d_3 (Conv1D)	(None, 9998, 64)	24640
conv1d_4 (Conv1D)	(None, 9995, 64)	16448
conv1d_5 (Conv1D)	(None, 9991, 64)	20544
global_max_pooling1d_1 (GlobalMaxPooling1D)	(None, 64)	0
reshape (Reshape)	(None, 1, 64)	0
lstm (LSTM)	(None, 128)	98816
dense_2 (Dense)	(None, 10)	1290

Total params: 1,441,738
 Trainable params: 1,441,738
 Non-trainable params: 0

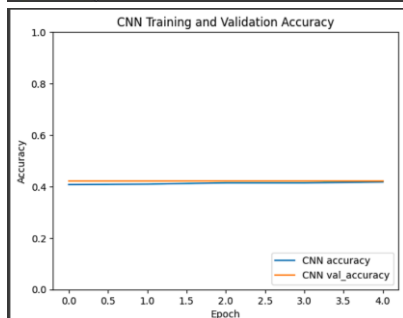


Test set
 Loss: 2.927
 Accuracy: 0.196

4. CNN vs Multilayer-LSTM

Model: "sequential"		
Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 10000, 128)	12800000
conv1d (Conv1D)	(None, 9998, 64)	24640
conv1d_1 (Conv1D)	(None, 9995, 64)	16448
conv1d_2 (Conv1D)	(None, 9991, 64)	20544
global_max_pooling1d (GlobalMaxPooling1D)	(None, 64)	0
dense (Dense)	(None, 128)	8320
dense_1 (Dense)	(None, 10)	1290

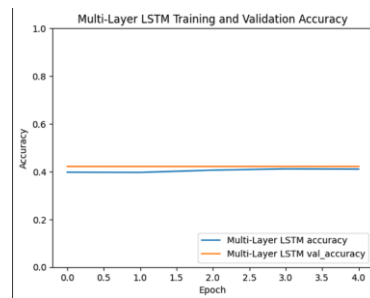
Total params: 1,351,242
 Trainable params: 1,351,242
 Non-trainable params: 0



Test set
 Loss: 2.858
 Accuracy: 0.196

Model: "sequential_8"		
Layer (type)	Output Shape	Param #
embedding_8 (Embedding)	(None, 5000, 128)	640000
lstm_9 (LSTM)	(None, 5000, 128)	131584
dropout_1 (Dropout)	(None, 5000, 128)	0
lstm_10 (LSTM)	(None, 5000, 128)	131584
dropout_2 (Dropout)	(None, 5000, 128)	0
lstm_11 (LSTM)	(None, 128)	131584
dropout_3 (Dropout)	(None, 128)	0
batch_normalization_8 (Batch Normalization)	(None, 128)	512
leaky_relu_8 (LeakyReLU)	(None, 128)	0
flatten_8 (Flatten)	(None, 128)	0
dense_8 (Dense)	(None, 10)	1290

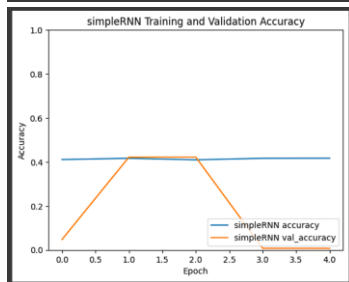
Total params: 1,036,554
 Trainable params: 1,036,298
 Non-trainable params: 256



Test set
 Loss: 2.342
 Accuracy: 0.196

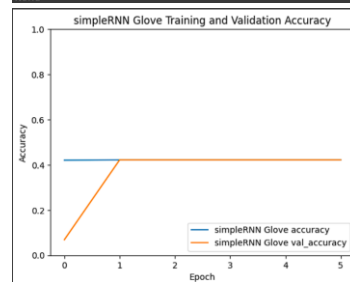
5. RNN vs. pre-trained Word Embedded RNN

Model: "sequential_6"		
Layer (type)	Output Shape	Param #
embedding_6 (Embedding)	(None, 5000, 128)	640000
simple_rnn_2 (SimpleRNN)	(None, 128)	32896
batch_normalization_6 (Batch Normalization)	(None, 128)	512
leaky_re_lu_6 (LeakyReLU)	(None, 128)	0
flatten_6 (Flatten)	(None, 128)	0
dense_6 (Dense)	(None, 10)	1290
Total params: 674,698		
Trainable params: 674,442		
Non-trainable params: 256		
None		



Test set
Loss: 2.858
Accuracy: 0.196

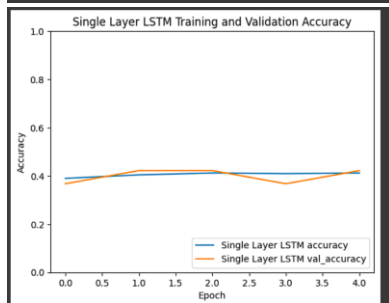
Model: "sequential_9"		
Layer (type)	Output Shape	Param #
embedding_9 (Embedding)	(None, 5000, 100)	500000
simple_rnn_3 (SimpleRNN)	(None, 128)	29312
batch_normalization_9 (Batch Normalization)	(None, 128)	512
leaky_re_lu_9 (LeakyReLU)	(None, 128)	0
flatten_9 (Flatten)	(None, 128)	0
dense_9 (Dense)	(None, 10)	1290
Total params: 531,114		
Trainable params: 30,858		
Non-trainable params: 500,256		
None		



Test set
Loss: 2.964
Accuracy: 0.196

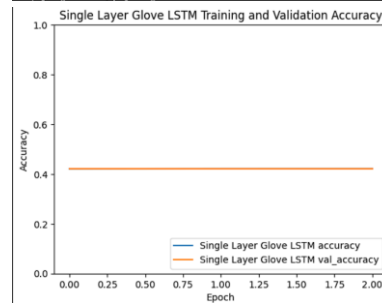
6. LSTM vs. pre-trained Word Embedded LSTM

Model: "sequential_7"		
Layer (type)	Output Shape	Param #
embedding_7 (Embedding)	(None, 5000, 128)	640000
lstm_8 (LSTM)	(None, 128)	131584
dropout (Dropout)	(None, 128)	0
batch_normalization_7 (Batch Normalization)	(None, 128)	512
leaky_re_lu_7 (LeakyReLU)	(None, 128)	0
flatten_7 (Flatten)	(None, 128)	0
dense_7 (Dense)	(None, 10)	1290
Total params: 773,386		
Trainable params: 773,130		
Non-trainable params: 256		
None		



Test set
Loss: 11.908
Accuracy: 0.110

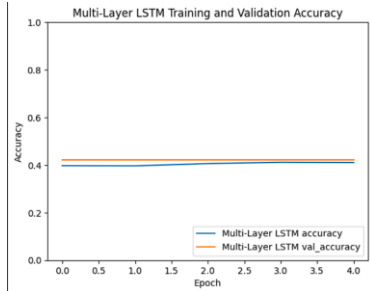
Model: "sequential_10"		
Layer (type)	Output Shape	Param #
embedding_10 (Embedding)	(None, 5000, 100)	500000
lstm_12 (LSTM)	(None, 128)	117248
batch_normalization_10 (Batch Normalization)	(None, 128)	512
leaky_re_lu_10 (LeakyReLU)	(None, 128)	0
flatten_10 (Flatten)	(None, 128)	0
dense_10 (Dense)	(None, 10)	1290
Total params: 619,050		
Trainable params: 118,794		
Non-trainable params: 500,256		
None		



Test set
Loss: 3.094
Accuracy: 0.196

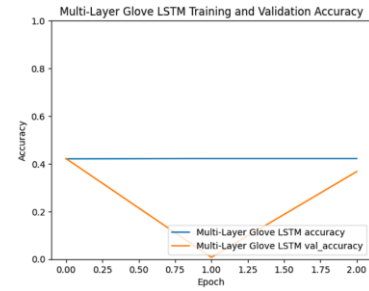
7. Multilayer-LSTM vs Pre-Trained Word embedding Multilayer-LSTM

Model: "sequential_8"		
Layer (type)	Output Shape	Param #
embedding_8 (Embedding)	(None, 5000, 128)	640000
lstm_9 (LSTM)	(None, 5000, 128)	131584
dropout_1 (Dropout)	(None, 5000, 128)	0
lstm_10 (LSTM)	(None, 5000, 128)	131584
dropout_2 (Dropout)	(None, 5000, 128)	0
lstm_11 (LSTM)	(None, 128)	131584
dropout_3 (Dropout)	(None, 128)	0
batch_normalization_8 (Batch Normalization)	(None, 128)	512
leaky_re_lu_8 (LeakyReLU)	(None, 128)	0
flatten_8 (Flatten)	(None, 128)	0
dense_8 (Dense)	(None, 10)	1290
Total params: 1,036,554		
Trainable params: 1,036,298		
Non-trainable params: 256		



Test set
Loss: 2.342
Accuracy: 0.196

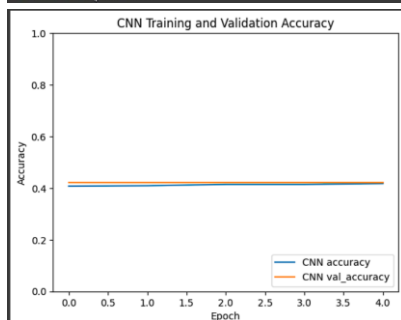
Model: "sequential_11"		
Layer (type)	Output Shape	Param #
embedding_11 (Embedding)	(None, 5000, 100)	500000
lstm_13 (LSTM)	(None, 5000, 128)	117248
lstm_14 (LSTM)	(None, 5000, 64)	49408
lstm_15 (LSTM)	(None, 64)	33024
batch_normalization_11 (Batch Normalization)	(None, 64)	256
leaky_re_lu_11 (LeakyReLU)	(None, 64)	0
flatten_11 (Flatten)	(None, 64)	0
dense_11 (Dense)	(None, 10)	650
Total params: 700,586		
Trainable params: 200,458		
Non-trainable params: 500,128		



Test set
Loss: 3.231
Accuracy: 0.152

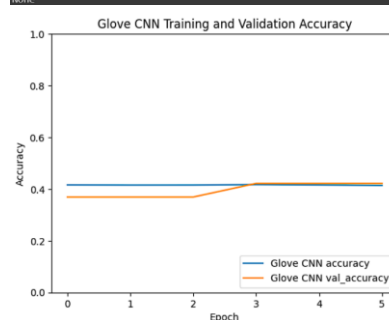
8. CNN vs. Pre-Trained Word Embedding CNN

Model: "sequential"		
Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 10000, 128)	1280000
conv1d (Conv1D)	(None, 9998, 64)	24640
conv1d_1 (Conv1D)	(None, 9995, 64)	16448
conv1d_2 (Conv1D)	(None, 9991, 64)	20544
global_max_pooling1d (Global Max Pooling1D)	(None, 64)	0
dense (Dense)	(None, 128)	8320
dense_1 (Dense)	(None, 10)	1290
Total params: 1,351,242		
Trainable params: 1,351,242		
Non-trainable params: 0		



Test set
Loss: 2.858
Accuracy: 0.196

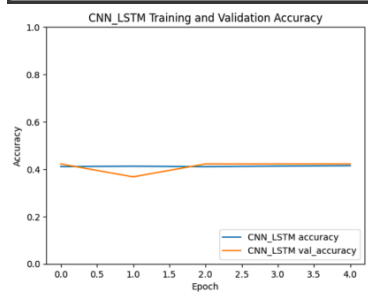
Model: "sequential_10"		
Layer (type)	Output Shape	Param #
embedding_10 (Embedding)	(None, 10000, 100)	1000000
conv1d_18 (Conv1D)	(None, 9998, 64)	19264
conv1d_19 (Conv1D)	(None, 9995, 64)	16448
conv1d_20 (Conv1D)	(None, 9991, 64)	20544
global_max_pooling1d_6 (Global Max Pooling1D)	(None, 64)	0
dense_12 (Dense)	(None, 128)	8320
dense_13 (Dense)	(None, 10)	1290
Total params: 1,065,866		
Trainable params: 65,866		
Non-trainable params: 1,000,000		



Test set
Loss: 3.027
Accuracy: 0.196

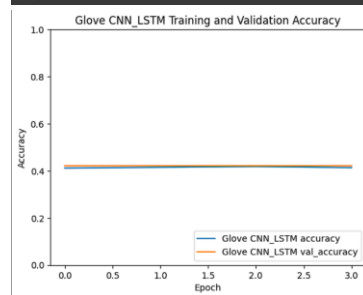
9. CNN as an additional layer to LSTM vs. Pre-Trained Word Embedded CNN as an additional layer to LSTM

Model: "sequential_1"		
Layer (type)	Output Shape	Param #
embedding_1 (Embedding)	(None, 10000, 128)	1280000
conv1d_3 (Conv1D)	(None, 9998, 64)	24640
conv1d_4 (Conv1D)	(None, 9995, 64)	16448
conv1d_5 (Conv1D)	(None, 9991, 64)	20544
global_max_pooling1d_1 (GlobalMaxPooling1D)	(None, 64)	0
reshape (Reshape)	(None, 1, 64)	0
lstm_1 (LSTM)	(None, 128)	98816
dense_2 (Dense)	(None, 10)	1290
Total params: 1,441,738		
Trainable params: 1,441,738		
Non-trainable params: 0		



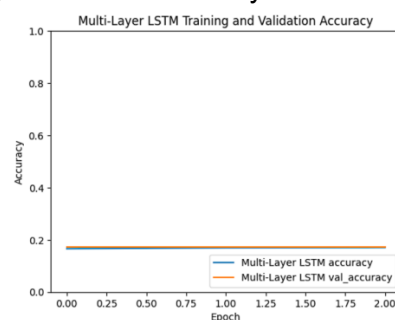
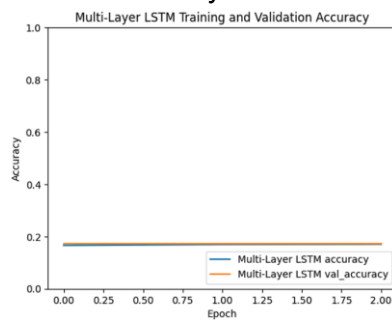
Test set
Loss: 2.927
Accuracy: 0.196

Model: "sequential_11"		
Layer (type)	Output Shape	Param #
embedding_11 (Embedding)	(None, 10000, 100)	1000000
conv1d_21 (Conv1D)	(None, 9998, 64)	19264
conv1d_22 (Conv1D)	(None, 9995, 64)	16448
conv1d_23 (Conv1D)	(None, 9991, 64)	20544
global_max_pooling1d_7 (GlobalMaxPooling1D)	(None, 64)	0
reshape_3 (Reshape)	(None, 1, 64)	0
lstm_7 (LSTM)	(None, 128)	98816
dense_14 (Dense)	(None, 10)	1290
Total params: 1,156,362		
Trainable params: 156,362		
Non-trainable params: 1,000,000		

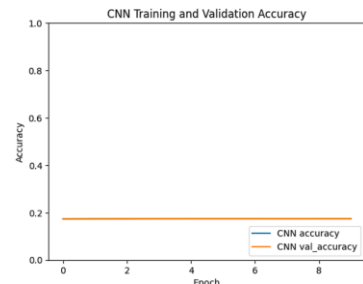
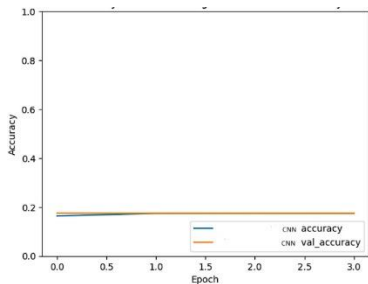


Test set
Loss: 2.871
Accuracy: 0.196

10. Multi-Layer LSTM Transfer Learning Model vs. Multi-Layer LSTM Model Ground up



11. CNN Transfer Learning Model vs. CNN Artist Prediction Ground up.



Discussion

Completing this assignment was both challenging and rewarding. One of the biggest obstacles I faced was the lack of time necessary to optimize the deep learning models efficiently. However, through the process of developing these models, I was able to gain a deeper understanding of important concepts such as activation functions, optimizers, data pre-processing, and loss functions. While it was a time-consuming task, the knowledge I gained from implementing these techniques was invaluable.

While some models were able to be trained relatively quickly, others required long hours to complete in the Google Colab environment. I have uploaded all the trained models to my Google Drive, where they can be accessed and evaluated in the demo notebook.

One of the most significant takeaways from this assignment was the importance of converting textual inputs into numerical values. This allowed me to explore a greater variety of models and save computational time. Ultimately, models such as Multi-Layer LSTM and CNN had the best accuracies. Additionally, using pre-trained data for word embedding improved the accuracy of the models.

Writing the lyrics component of the assignment was both challenging and fascinating. Although I know that I could have done a better job with more time, I am proud of the work that I was able to produce within the given timeframe. In the future, I hope to explore the role of deep learning in the context of text generation, particularly with the advent of ChatGPT and the potential for even more advanced General Artificial Intelligence. Overall, I am extremely pleased with the insights and experience I gained during this semester. These skills will be invaluable as I continue to pursue computer science and apply neural networks to my future projects and thesis.